DATABASE DESIGN · VMS

Database Design Proposal

For review

The proposed tables, columns, keys, constraints and relationships for the VMS database.

PURPOSE	DETAIL LEVEL	STATUS
Present the proposed database design for review and discussion.	Columns, data types, keys, constraints and indexes for every table.	Detailed proposal from the ideation phase. 37 tables; 72 foreign keys.

DESIGN IDEA

Keep a person's identity separate from their login account, a visit request, the actual visit and the kiosk pass, so `people`, `users`, `invitations`, `visits` and `passes` are distinct tables. The VMS is intended to serve our own organisation across one or more of its sites.

Keys, defaults, constraints and indexes shown below are proposed as database-level rules. Rules intended to be handled by the application instead are labelled **Application rule**.

1 Overall structure

The central records and how a visit is intended to flow through them.

PEOPLE
Person identity
May connect to a user or host

→

INVITATIONS
Request before arrival
Leads to at most one visit

→

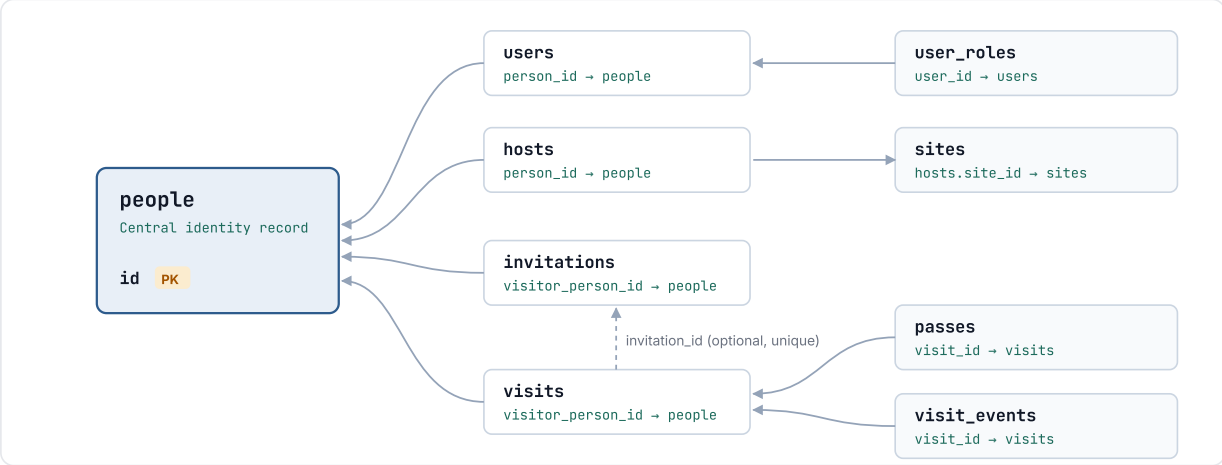
VISITS
Actual visit and status
Has visit events

→

PASSES
QR/pass validity
At most one per visit

How the tables connect to `people`

Arrows point from the table holding the foreign key to the table it references. Only the main links are drawn; site, host and audit references are omitted.



Main table inventory

TABLE	AREA	PURPOSE	SECTION
sites	SITES	Lists the organisation's sites, such as offices or buildings, where visits can take place.	§2.1
site_settings	SITES	Holds the visitor rules for each site, such as the default visit length and whether walk-ins need host approval.	§2.2
departments	SITES	Lists the departments or teams that hosts belong to.	§2.3
gates	SITES	Lists the named entrances at each site, such as a main lobby or a loading bay.	§2.4
devices	SITES	Records each kiosk or scanner registered at a site, together with the secure codes used to pair it.	§2.5
people	IDENTITY	Stores the basic details of each person known to the VMS, such as their name and contact information.	§2.6
person_contacts	IDENTITY	Stores each email address or phone number for a person and whether it has been confirmed.	§2.7
visitor_profiles	IDENTITY	Keeps a long-term record for each visitor, such as how often they have visited and whether they are allowed in.	§2.8
person_merge_log	IDENTITY	Records when two records for the same visitor are combined into one, who did it and why.	§2.9
users	AUTH	Stores the login account for each staff member who uses the dashboard.	§2.10
user_credentials	AUTH	Stores the securely hashed password for each login account.	§2.11
roles	AUTH	Lists the available staff roles, such as receptionist or site administrator.	§2.12
permissions	AUTH	Lists the individual actions the system can allow or block, such as approving a visit.	§2.13
role_permissions	AUTH	Links each role to the permissions it grants.	§2.14
user_roles	AUTH	Records which role each staff member has and at which sites.	§2.15
user_sessions	AUTH	Keeps a record of each active sign-in.	§2.16
password_reset_requests	AUTH	Tracks the password reset links that have been sent, including when they expire and whether they have been used.	§2.17
authentication_events	AUTH	Keeps a running log of sign-ins, failed attempts, lockouts and password resets.	§2.18
hosts	VISITS	Lists the employees that visitors can come to see, with their employee ID, department and site.	§2.19

TABLE	AREA	PURPOSE	SECTION
`invitations`	VISITS	Stores visits that a host arranges in advance, before the visitor arrives.	§2.20
`visits`	VISITS	Stores each actual visit: who is visiting whom, at which site, when, and its current status.	§2.21
`visit_events`	VISITS	Keeps a step-by-step history of each visit, such as approval, check-in, stepping out and check-out.	§2.22
`passes`	VISITS	Stores the QR pass issued for a visit and how long it is valid.	§2.23
`pass_scans`	VISITS	Records every time a pass is scanned or checked, and whether it was accepted or why it was refused.	§2.24
`idempotency_keys`	VISITS	Remembers recent requests so that a request sent twice, for example after a network drop at a kiosk, creates only one visit.	§2.25
`blacklist_entries`	SECURITY	Lists people who should be refused entry or need security approval, with the reason and how long the restriction lasts.	§2.26
`blacklist_matches`	SECURITY	Records when a visitor appears to match a restricted-entry record and how certain the match is.	§2.27
`security_approvals`	SECURITY	Records the security team's decision about a visitor who matched a restricted-entry record.	§2.28
`audit_logs`	AUDIT	Keeps a permanent record of important actions, such as who approved a visit or changed an account.	§2.29
`notifications`	AUDIT	Stores messages sent to staff, such as "your visitor has arrived", and whether each was delivered.	§2.30
`notification_attempts`	AUDIT	Records each attempt to send an email or text message and its result.	§2.31
`outbox_events`	AUDIT	Holds a queue of follow-up work created when a record is saved, such as emails to send.	§2.32
`retention_policies`	OPS	Sets how long each kind of record is kept and whether it is deleted or anonymised afterwards.	§2.33
`retention_holds`	OPS	Marks records that must be kept beyond the normal retention period, for example during a legal case or an investigation.	§2.34
`visit_daily_stats`	OPS	Keeps daily visit totals without any personal details.	§2.35
`job_runs`	OPS	Records each run of a background task, such as the nightly clean-up, and whether it succeeded.	§2.36
`backup_runs`	OPS	Records each database backup and each test restore, and whether it succeeded.	§2.37

All foreign keys

Every proposed foreign key, with its cardinality (derived from primary keys and unique constraints) and intended delete behaviour. Many-to-many relationships use the junction tables `role_permissions` (roles ↔ permissions) and `user_roles` (users ↔ roles, per site).

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
site_settings	site_id	sites	one-to-one	cascade
gates	site_id	sites	many-to-one	restrict
devices	enrolled_by	users	many-to-one · optional	restrict
devices	gate_id	gates	many-to-one · optional	restrict
devices	site_id	sites	many-to-one	restrict
people	created_by	users	many-to-one · optional	set null
people	merged_into_id	people	many-to-one · optional	restrict
people	updated_by	users	many-to-one · optional	set null
person_contacts	person_id	people	many-to-one	cascade
visitor_profiles	person_id	people	one-to-one	restrict
person_merge_log	merged_by	users	many-to-one	restrict
person_merge_log	source_person_id	people	many-to-one	restrict
person_merge_log	target_person_id	people	many-to-one	restrict
users	person_id	people	one-to-one	restrict
user_credentials	user_id	users	one-to-one	cascade
role_permissions	permission_id	permissions	many-to-one	cascade
role_permissions	role_id	roles	many-to-one	cascade
user_roles	site_id	sites	many-to-one · optional	restrict
user_roles	user_id	users	many-to-one	cascade
user_roles	granted_by	users	many-to-one · optional	set null
user_roles	role_id	roles	many-to-one	restrict
user_sessions	user_id	users	many-to-one	cascade
password_reset_requests	contact_id	person_contacts	many-to-one	cascade
password_reset_requests	initiated_by	users	many-to-one · optional	set null
password_reset_requests	user_id	users	many-to-one; unique where (consumed_at IS NULL) AND (cancelled_at IS NULL)	cascade
authentication_events	user_id	users	many-to-one · optional	set null
hosts	department_id	departments	many-to-one	restrict

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
hosts	person_id	people	many-to-one	restrict
hosts	site_id	sites	many-to-one	restrict
hosts	user_id	users	one-to-one · optional	restrict
invitations	host_id	hosts	many-to-one	restrict
invitations	site_id	sites	many-to-one	restrict
invitations	visitor_person_id	people	many-to-one	restrict
invitations	cancelled_by	users	many-to-one · optional	restrict
invitations	created_by	users	many-to-one · optional	restrict
visits	created_by_device_id	devices	many-to-one · optional	restrict
visits	host_id	hosts	many-to-one	restrict
visits	invitation_id	invitations	one-to-one · optional	restrict
visits	site_id	sites	many-to-one	restrict
visits	visitor_person_id	people	many-to-one	restrict
visits	created_by_id	users	many-to-one · optional	restrict
visit_events	pass_id	passes	many-to-one · optional	restrict
visit_events	visit_id	visits	many-to-one	restrict
visit_events	actor_user_id	users	many-to-one · optional	restrict
visit_events	device_id	devices	many-to-one · optional	restrict
passes	visit_id	visits	one-to-one	restrict
passes	issued_by_device_id	devices	many-to-one · optional	restrict
passes	issued_by_user_id	users	many-to-one · optional	restrict
passes	revoked_by	users	many-to-one · optional	restrict
pass_scans	actor_user_id	users	many-to-one · optional	restrict
pass_scans	device_id	devices	many-to-one · optional	restrict
pass_scans	pass_id	passes	many-to-one · optional	restrict
pass_scans	site_id	sites	many-to-one · optional	restrict
blacklist_entries	created_by	users	many-to-one	restrict
blacklist_entries	lifted_by	users	many-to-one · optional	restrict
blacklist_entries	person_id	people	many-to-one · optional	restrict
blacklist_entries	site_id	sites	many-to-one · optional	restrict
blacklist_matches	blacklist_entry_id	blacklist_entries	many-to-one	restrict

TABLE	COLUMN(S)	REFERENCES	CARDINALITY	ON DELETE
blacklist_matches	reviewed_by	users	many-to-one · optional	restrict
blacklist_matches	visit_id	visits	many-to-one	restrict
security_approvals	visit_id	visits	many-to-one; unique where status = 'pending'	restrict
security_approvals	decided_by	users	many-to-one · optional	restrict
audit_logs	actor_device_id	devices	many-to-one · optional	restrict
audit_logs	actor_user_id	users	many-to-one · optional	restrict
notifications	recipient_id	users	many-to-one	restrict
notifications	visit_id	visits	many-to-one · optional	set null
notification_attempts	notification_id	notifications	many-to-one	cascade
retention_policies	site_id	sites	many-to-one · optional	restrict
retention_policies	approved_by	users	many-to-one · optional	restrict
retention_holds	placed_by	users	many-to-one	restrict
retention_holds	released_by	users	many-to-one · optional	restrict
visit_daily_stats	site_id	sites	many-to-one	restrict

2 Database tables

One section per table, in the order of the inventory. Each lists the proposed columns, data types, keys and defaults, followed by the proposed constraints, indexes and triggers.

PK Primary key FK → table Foreign key and the table it references UUID Unique identifier TIMESTAMPTZ Date and time with timezone

2.1 sites

SITES

PURPOSE Lists the organisation's sites, such as offices or buildings, where visits can take place. Visits, hosts, kiosks and staff access are all tied to a site.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
name	VARCHAR(180)	Unique Required	Display name of the site.
address	VARCHAR(300)	Optional	Physical address.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
status	TEXT	Required · default `'active'`	active, inactive or archived.

is_active	BOOLEAN	Generated: status = 'active'	True when status is active (generated column).
code	VARCHAR(40)	Optional	Optional short site code.
timezone	VARCHAR(64)	Required · default `'UTC'`	IANA timezone name, used for display.
archived_at	TIMESTAMPTZ	Optional	When the site was archived.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.

CONSTRAINT	DEFINITION	NAME
Unique index	`lower((code)) WHERE code IS NOT NULL`	`uq_sites_code`
Check	`(status = 'archived') = (archived_at IS NOT NULL)`	`ck_sites_archived_at`
Check	`status IN ('active', 'inactive', 'archived')`	`ck_sites_status`
Trigger	`set_updated_at()`	`trg_sites_updated_at`

2.2 site_settings

SITES

PURPOSE Holds the visitor rules for each site, such as the default visit length and whether walk-ins need host approval. This lets each site work a little differently without code changes.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
site_id	UUID	PK FK → sites *on delete cascade* Required	The site these settings belong to.
operating_hours	JSONB	Required · default `'{}'`	Opening hours (JSON).
default_visit_hours	INTEGER	Required · default `8`	Default visit length for walk-ins.
walk_in_requires_host_approval	BOOLEAN	Required · default `true`	Whether kiosk walk-ins wait for host approval.
overdue_grace_minutes	INTEGER	Required · default `30`	Minutes after the visit end before a visitor is escalated as overdue.
allow_reentry	BOOLEAN	Required · default `true`	Whether visitors may step out and re-enter on the same pass.
require_nda	BOOLEAN	Required · default `true`	Whether visitors must accept the NDA.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(default_visit_hours ⩾ 1) AND (default_visit_hours ⩽ 72)`	`ck_site_settings_default_visit_hours`
Check	`(overdue_grace_minutes ⩾ 0) AND (overdue_grace_minutes ⩽ 1440)`	`ck_site_settings_overdue_grace`
Trigger	`set_updated_at()`	`trg_site_settings_updated_at`

Application rule. The application is intended to create the settings row together with each site.

2.3 departments

SITES

PURPOSE Lists the departments or teams that hosts belong to. Visitors and reception can then find a host by department.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
name	VARCHAR(120)	Required	Department name.
status	TEXT	Required · default `'active'`	active or inactive.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique index	`lower((name))`	`uq_departments_name`
Check	`length(TRIM(BOTH FROM name)) > 0`	`ck_departments_name_not_blank`
Check	`status IN ('active', 'inactive')`	`ck_departments_status`
Trigger	`set_updated_at()`	`trg_departments_updated_at`

2.4 gates

SITES

PURPOSE Lists the named entrances at each site, such as a main lobby or a loading bay. Kiosks and scanners can then be linked to the entrance they serve.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
name	VARCHAR(120)	Required	Gate name.
status	TEXT	Required · default `'active'`	active or inactive.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique	`site_id, name`	`uq_gates_site_name`
Check	`status IN ('active', 'inactive')`	`ck_gates_status`

CONSTRAINT	DEFINITION	NAME
Trigger	set_updated_at()	trg_gates_updated_at

2.5 devices

SITES

PURPOSE Records each kiosk or scanner registered at a site, together with the secure codes used to pair it. Only registered devices are meant to be able to check visitors in and out.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default gen_random_uuid()	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
gate_id	UUID	FK → gates *on delete restrict* Optional	Gate the device is mounted at, if any.
name	VARCHAR(120)	Required	Device name, unique per site.
device_type	TEXT	Required	kiosk, scanner or badge_printer.
credential_hash	VARCHAR(64)	Unique Unique where credential_hash IS NOT NULL Optional	Keyed hash of the device key (set when paired).
status	TEXT	Required · default 'pending'	pending, active, disabled or retired.
enrolled_at	TIMESTAMPTZ	Optional	When the device was paired.
enrolled_by	UUID	FK → users *on delete restrict* Optional	Account that registered the device.
last_seen_at	TIMESTAMPTZ	Optional	Last authenticated request from the device.
created_at	TIMESTAMPTZ	Required · default now()	When the row was created.
updated_at	TIMESTAMPTZ	Required · default now()	Last change; maintained by trigger.
pairing_code_hash	VARCHAR(64)	Unique Unique where pairing_code_hash IS NOT NULL Optional	Keyed hash of the one-time pairing code.
pairing_expires_at	TIMESTAMPTZ	Optional	When the pairing code expires.

CONSTRAINT	DEFINITION	NAME
Unique	site_id, name	uq_devices_site_name

CONSTRAINT	DEFINITION	NAME
Unique index	credential_hash WHERE credential_hash IS NOT NULL	uq_devices_credential_hash
Unique index	pairing_code_hash WHERE pairing_code_hash IS NOT NULL	uq_devices_pairing_code_hash
Check	(status <> 'active') OR ((credential_hash IS NOT NULL) AND (enrolled_at IS NOT NULL))	ck_devices_active_enrolled
Check	status IN ('pending', 'active', 'disabled', 'retired')	ck_devices_status
Check	device_type IN ('kiosk', 'scanner', 'badge_printer')	ck_devices_type
Index	btree (site_id) WHERE status = 'active'	ix_devices_site_active
Trigger	set_updated_at()	trg_devices_updated_at

Application rule. Pairing codes are intended to expire after 15 minutes and work only once; device keys are hashed with a server-side key. Both are handled by the application.

2.6 people

IDENTITY

PURPOSE Stores the basic details of each person known to the VMS, such as their name and contact information. Visitors, hosts and staff all refer to this table instead of repeating the same person's details.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
full_name	VARCHAR(160)	Required	Person's name.
email	VARCHAR(320)	Optional	Contact email (stored lowercase; not unique).
phone	VARCHAR(40)	Optional	Contact number (not unique).
company	VARCHAR(160)	Optional	Company the visitor represents.
identity_status	TEXT	Required · default `'active'`	active, merged or anonymized.
merged_into_id	UUID	FK → people *on delete restrict* Optional	Surviving record when this one was merged.
anonymized_at	TIMESTAMPTZ	Optional	When personal details were removed.
created_by	UUID	FK → users *on delete set null* Optional	Account that created the record.
updated_by	UUID	FK → users *on delete set null* Optional	Account that last changed the record.

version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(identity_status = 'anonymized') = (anonymized_at IS NOT NULL)`	`ck_people_anonymized_at`
Check	`(email IS NULL) OR (email = lower((email)))`	`ck_people_email_lowercase`
Check	`length(TRIM(BOTH FROM full_name)) > 0`	`ck_people_full_name_not_blank`
Check	`identity_status IN ('active', 'merged', 'anonymized')`	`ck_people_identity_status`
Check	`(identity_status = 'merged') = (merged_into_id IS NOT NULL)`	`ck_people_merged_into`
Index	`gin (full_name (trigram))`	`ix_people_full_name_trgm`
Index	`btree (lower((email))) WHERE email IS NOT NULL`	`ix_people_email`
Index	`btree (phone) WHERE phone IS NOT NULL`	`ix_people_phone`
Index	`btree ("right"(regexp_replace((phone), '[^0-9]', '', 'g'), 10)) WHERE phone IS NOT NULL`	`ix_people_phone_key`
Trigger	`set_updated_at()`	`trg_people_updated_at`

> **Application rule.** Matching a returning visitor (same email or phone and a similar name) and merging duplicates are intended to be handled by the application, not by database rules. A name alone never links two people.

2.7 person_contacts

IDENTITY

PURPOSE Stores each email address or phone number for a person and whether it has been confirmed. Confirmed contacts are needed for password resets and notifications.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
person_id	UUID	FK → people *on delete cascade* Required	Person the channel belongs to.
channel	TEXT	Required	email or phone.
value	VARCHAR(320)	Required	Email address or phone number.
verified_at	TIMESTAMPTZ	Optional	When control of the channel was proven.
is_primary	BOOLEAN	Required · default `false`	The primary channel of its type.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique	`person_id, channel, value`	`uq_person_contacts_value`
Unique index	`person_id, channel WHERE is_primary`	`uq_person_contacts_primary`
Check	`channel IN ('email', 'phone')`	`ck_person_contacts_channel`
Check	`(channel <> 'email') OR (value = lower((value)))`	`ck_person_contacts_email_lowercase`
Trigger	`set_updated_at()`	`trg_person_contacts_updated_at`

2.8 visitor_profiles

IDENTITY

PURPOSE Keeps a long-term record for each visitor, such as how often they have visited and whether they are allowed in. This lets staff recognise returning visitors and block someone if necessary.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
person_id	UUID	PK FK → people *on delete restrict* Required	Person this profile belongs to.
status	TEXT	Required · default `'active'`	active, or inactive (blocks entry).
classification	TEXT	Required · default `'standard'`	standard, frequent, contractor, vip or restricted.
identity_review	BOOLEAN	Required · default `false`	Flagged as a possible duplicate for staff review.
first_visit_at	TIMESTAMPTZ	Optional	First check-in.
last_visit_at	TIMESTAMPTZ	Optional	Most recent check-in.
visit_count	INTEGER	Required · default `0`	Number of visits with a check-in.
notes	VARCHAR(1000)	Optional	Administrator notes.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`classification IN ('standard', 'frequent', 'contractor', 'vip', 'restricted')`	`ck_visitor_profiles_classification`
Check	`status IN ('active', 'inactive')`	`ck_visitor_profiles_status`
Check	`visit_count ⩾ 0`	`ck_visitor_profiles_visit_count`
Index	`btree (classification)`	`ix_visitor_profiles_classification`
Index	`btree (last_visit_at DESC NULLS LAST)`	`ix_visitor_profiles_last_visit`
Trigger	`set_updated_at()`	`trg_visitor_profiles_updated_at`

2.9 person_merge_log

IDENTITY

PURPOSE Records when two records for the same visitor are combined into one, who did it and why. This keeps a trail of every merge of duplicate records.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
source_person_id	UUID	FK → people *on delete restrict* Required	Duplicate record that was merged away.
target_person_id	UUID	FK → people *on delete restrict* Required	Record that was kept.
merged_by	UUID	FK → users *on delete restrict* Required	Administrator who merged.
reason	VARCHAR(500)	Required	Reason given.
moved_counts	JSONB	Required	How many visits and invitations moved (JSON).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

CONSTRAINT	DEFINITION	NAME
Check	`source_person_id <> target_person_id`	`ck_person_merge_distinct`

2.10 users

AUTH

PURPOSE Stores the login account for each staff member who uses the dashboard. Login details are kept apart from personal details because not every person needs an account.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
email	VARCHAR(320)	Unique Required	Login email (stored lowercase).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
person_id	UUID	FK → people *on delete restrict* Unique Required	Person associated with the account.
account_status	TEXT	Required · default `'active'`	invited, active, locked or disabled.
last_login_at	TIMESTAMPTZ	Optional	Last successful sign-in.

failed_login_count	INTEGER	Required · default `0`	Consecutive failed sign-ins.
locked_until	TIMESTAMPTZ	Optional	Sign-in lockout end, if locked.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
is_active	BOOLEAN	Generated: account_status = 'active'	True when account_status is active (generated column).

CONSTRAINT	DEFINITION	NAME
Unique index	`email`	`ix_users_email`
Check	`account_status IN ('invited', 'active', 'locked', 'disabled')`	`ck_users_account_status`
Check	`failed_login_count ⩾ 0`	`ck_users_failed_login_count`
Trigger	`set_updated_at()`	`trg_users_updated_at`

> **Application rule.** The application is intended to store emails in lower case and check for duplicates; the database unique index on email is case-sensitive.

2.11 user_credentials

AUTH

PURPOSE Stores the securely hashed password for each login account. Keeping it in its own table limits where password data can appear.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
user_id	UUID	PK FK → users *on delete cascade* Required	Account the credential belongs to.
password_hash	TEXT	Required	Argon2id hash, never the password.
password_changed_at	TIMESTAMPTZ	Required · default `now()`	When the password was last set.
must_change	BOOLEAN	Required · default `false`	Whether a change is required at next sign-in.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`password_hash ~ '$argon2id$%'`	`ck_user_credentials_argon2id`

CONSTRAINT	DEFINITION	NAME
Trigger	set_updated_at()	trg_user_credentials_updated_at

Application rule. Hashing with Argon2id is done by the application; the database only checks that the stored value is an Argon2id hash.

2.12 roles

AUTH

PURPOSE Lists the available staff roles, such as receptionist or site administrator. Roles decide what each staff member is allowed to do.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	SMALLINT	PK Required · default nextval('roles_id_seq'::regclass)	Role identifier.
code	VARCHAR(40)	Unique Required	super-admin, site-admin, receptionist or host.
scope	TEXT	Required	global, global_or_site or site.
priority	SMALLINT	Required	Picks the effective role when an account holds several.

CONSTRAINT	DEFINITION	NAME
Check	scope IN ('global', 'global_or_site', 'site')	ck_roles_scope

2.13 permissions

AUTH

PURPOSE Lists the individual actions the system can allow or block, such as approving a visit. The API checks these before carrying out an action.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	SMALLINT	PK Required · default nextval('permissions_id_seq'::regclass)	Permission identifier.
code	VARCHAR(60)	Unique Required	Permission code, e.g. visit.approve.

2.14 role_permissions

AUTH

PURPOSE Links each role to the permissions it grants. This makes it possible to change what a role can do without changing code.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
role_id	SMALLINT	PK *composite* FK → roles *on delete cascade* Required	Role.
permission_id	SMALLINT	PK *composite* FK → permissions *on delete cascade* Required	Permission granted to the role.

2.15 user_roles

AUTH

PURPOSE Records which role each staff member has and at which sites. This is how the system knows that a receptionist works at one site but not another.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Required	Account receiving the role.
role_id	SMALLINT	FK → roles *on delete restrict* Required	Role assigned.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the role applies to; empty means all sites.
granted_by	UUID	FK → users *on delete set null* Optional	Account that granted the role.
granted_at	TIMESTAMPTZ	Required · default `now()`	When it was granted.

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id, role_id, COALESCE(site_id, '00000000-0000-0000-0000-000000000000')`	`uq_user_roles_assignment`
Index	`btree (site_id)`	`ix_user_roles_site`
Trigger	`enforce_user_role_scope()`	`trg_user_roles_scope`

Application rule. Choosing the effective role (highest priority) and the sites it covers is intended to be handled by the application.

2.16 user_sessions

AUTH

PURPOSE Keeps a record of each active sign-in. This allows a sign-in to be ended straight away, for example on logout or after a password change.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Required	Account signed in.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the session ends.
revoked_at	TIMESTAMPTZ	Optional	When it was revoked (logout, password change, deactivation).
revoke_reason	VARCHAR(40)	Optional	Why it was revoked.
user_agent	VARCHAR(300)	Optional	Browser user agent.
ip	INET	Optional	Client IP address.

CONSTRAINT	DEFINITION	NAME
Check	`expires_at > created_at`	`ck_user_sessions_expiry`
Check	`(revoked_at IS NULL) = (revoke_reason IS NULL)`	`ck_user_sessions_revoke_reason`
Index	`btree (user_id) WHERE revoked_at IS NULL`	`ix_user_sessions_user_active`

Application rule. The API is intended to reject any request whose session has been revoked or has expired.

2.17 password_reset_requests

AUTH

PURPOSE Tracks the password reset links that have been sent, including when they expire and whether they have been used. This makes sure each link works only once and only for a short time.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
user_id	UUID	FK → users *on delete cascade* Unique where (consumed_at IS NULL) AND (cancelled_at IS NULL) Required	Account being reset.
channel	TEXT	Required	email or sms.
contact_id	UUID	FK → person_contacts *on delete cascade* Required	Contact the link was sent to.
token_hash	VARCHAR(64)	Unique Required	Keyed hash of the reset token.
initiated_by	UUID	FK → users *on delete set null* Optional	Administrator who started it, if not self-service.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the link expires.
attempt_count	INTEGER	Required · default `0`	Attempts used.
max_attempts	INTEGER	Required · default `5`	Attempts allowed.
consumed_at	TIMESTAMPTZ	Optional	When the link was used.
cancelled_at	TIMESTAMPTZ	Optional	When a newer request cancelled it.
request_ip	INET	Optional	IP address of the request.

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id WHERE (consumed_at IS NULL) AND (cancelled_at IS NULL)`	`uq_password_reset_live`
Check	`(attempt_count ⩾ 0) AND (attempt_count ⩽ max_attempts)`	`ck_password_reset_attempts`
Check	`channel IN ('email', 'sms')`	`ck_password_reset_channel`
Check	`expires_at > created_at`	`ck_password_reset_expiry`
Check	`NOT ((consumed_at IS NOT NULL) AND (cancelled_at IS NOT NULL))`	`ck_password_reset_single_outcome`

CONSTRAINT	DEFINITION	NAME
Index	btree (user_id, created_at DESC)	ix_password_reset_user_created

> **Application rule.** Expiry, single use (through a conditional update) and rate limits are intended to be enforced by the application.

2.18 authentication_events

AUTH

PURPOSE Keeps a running log of sign-ins, failed attempts, lockouts and password resets. This helps to detect misuse and investigate account problems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Unique identifier.
user_id	UUID	FK → users *on delete set null* Optional	Account involved, if known.
event_type	VARCHAR(40)	Required	e.g. login.success, login.failure, reset.completed.
ip	INET	Optional	Client IP address.
user_agent	VARCHAR(300)	Optional	Browser user agent.
metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).
occurred_at	TIMESTAMPTZ	Required · default `now()`	When it happened.

CONSTRAINT	DEFINITION	NAME
Index	btree (ip, event_type, occurred_at DESC)	ix_authentication_events_ip_type
Index	btree (occurred_at)	ix_authentication_events_occurred
Index	btree (user_id, occurred_at DESC)	ix_authentication_events_user
Trigger	reject_history_change()	trg_authentication_events_append_only

2.19 hosts

VISITS

PURPOSE Lists the employees that visitors can come to see, with their employee ID, department and site. Each host links to the person's details and, if they have one, their login account.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
employee_id	VARCHAR(80)	Unique Required	Internal employee identifier.
person_id	UUID	FK → people *on delete restrict* Required	Person who acts as the host.
user_id	UUID	FK → users *on delete restrict* Unique Unique where user_id IS NOT NULL Optional	Login account of the host, if any.
department_id	UUID	FK → departments *on delete restrict* Required	Department.
status	TEXT	Required · default `'active'`	active or inactive.
deactivated_at	TIMESTAMPTZ	Optional	When the host was deactivated.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
active	BOOLEAN	Generated: status = 'active'	True when status is active (generated column).

CONSTRAINT	DEFINITION	NAME
Unique index	`user_id WHERE user_id IS NOT NULL`	`uq_hosts_user_id`
Check	`(status = 'inactive') = (deactivated_at IS NOT NULL)`	`ck_hosts_deactivated_at`
Check	`status IN ('active', 'inactive')`	`ck_hosts_status`
Index	`gin (employee_id (trigram))`	`ix_hosts_employee_id_trgm`
Index	`btree (site_id)`	`ix_hosts_site_id`

CONSTRAINT	DEFINITION	NAME
Index	`btree (site_id, status)`	`ix_hosts_site_status`
Trigger	`set_updated_at()`	`trg_hosts_updated_at`

> **Application rule.** Employee IDs are also intended to be checked case-insensitively by the application; the database unique constraint is case-sensitive.

2.20 invitations

VISITS

PURPOSE Stores visits that a host arranges in advance, before the visitor arrives. This keeps the original request separate from what actually happens on the day.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
host_id	UUID	FK → hosts *on delete restrict* Required	Host being visited.
visitor_person_id	UUID	FK → people *on delete restrict* Required	Person expected to visit.
purpose	VARCHAR(500)	Required	Reason for the visit.
scheduled_start	TIMESTAMPTZ	Required	Expected start.
scheduled_end	TIMESTAMPTZ	Required	Expected end.
status	TEXT	Required · default `'pending'`	pending, approved, rejected, cancelled or expired.
created_by	UUID	FK → users *on delete restrict* Optional	Account that created it.
cancelled_by	UUID	FK → users *on delete restrict* Optional	Account that cancelled it.
cancelled_at	TIMESTAMPTZ	Optional	When it was cancelled.
cancel_reason	VARCHAR(500)	Optional	Reason given.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.

created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(status = 'cancelled') = (cancelled_at IS NOT NULL)`	`ck_invitations_cancelled_at`
Check	`scheduled_end > scheduled_start`	`ck_invitations_period`
Check	`status IN ('pending', 'approved', 'rejected', 'cancelled', 'expired')`	`ck_invitations_status`
Index	`btree (host_id, scheduled_start DESC)`	`ix_invitations_host_start`
Index	`btree (site_id, status, scheduled_start)`	`ix_invitations_site_status_start`
Index	`btree (visitor_person_id)`	`ix_invitations_visitor`
Trigger	`reject_inactive_host()`	`trg_invitations_active_host`
Trigger	`set_updated_at()`	`trg_invitations_updated_at`

> **Application rule.** The application is intended to keep the invitation status in step with its visit.

2.21 visits

VISITS

PURPOSE Stores each actual visit: who is visiting whom, at which site, when, and its current status. It is the main record staff work with day to day.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Required	Site the row belongs to.
visitor_person_id	UUID	FK → people *on delete restrict* Required	Visitor for this visit.
host_id	UUID	FK → hosts *on delete restrict* Required	Host being visited.
created_by_id	UUID	FK → users *on delete restrict* Optional	Staff account that created it.
purpose	VARCHAR(500)	Required	Reason for the visit.
starts_at	TIMESTAMPTZ	Required	Expected start.

ends_at	TIMESTAMPTZ	Required	Expected end.
status	TEXT	Required	pending, security-review, scheduled, checked-in, temporarily-out, checked-out, no-show, cancelled or rejected.
source	VARCHAR(24)	Required · default `'invited'`	invited or walk-in.
checked_in_at	TIMESTAMPTZ	Optional	First entry.
checked_out_at	TIMESTAMPTZ	Optional	Final exit.
check_in_gate	VARCHAR(120)	Optional	Label of the entry point used.
privacy_consent_at	TIMESTAMPTZ	Optional	When the privacy notice was accepted.
nda_signed_at	TIMESTAMPTZ	Optional	When the NDA was accepted.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.
invitation_id	UUID	**FK → invitations** *on delete restrict* Unique Unique where invitation_id IS NOT NULL Optional	Invitation that led to the visit, if any.
category	TEXT	Required · default `'business'`	business, interview, contractor, delivery, personal or other.
created_by_device_id	UUID	**FK → devices** *on delete restrict* Optional	Kiosk that created it, if any.
overdue_notified_at	TIMESTAMPTZ	Optional	When overdue escalation was sent.
retention_eligible_at	TIMESTAMPTZ	Optional	Start of the retention clock (set at a terminal status).
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
anonymized_at	TIMESTAMPTZ	Optional	When retention removed personal details.

CONSTRAINT	DEFINITION	NAME
Unique index	`invitation_id WHERE invitation_id IS NOT NULL`	`uq_visits_invitation`
Check	`category IN ('business', 'interview', 'contractor', 'delivery', 'personal', 'other')`	`ck_visits_category`
Check	`status NOT IN ('checked-in', 'temporarily-out', 'checked-out') OR (checked_in_at IS NOT NULL)`	`ck_visits_checked_in`

CONSTRAINT	DEFINITION	NAME
Check	(status <> 'checked-out') OR (checked_out_at IS NOT NULL)	ck_visits_checked_out
Check	ends_at > starts_at	ck_visits_period
Check	(retention_eligible_at IS NULL) OR status IN ('checked-out', 'no-show', 'cancelled', 'rejected')	ck_visits_retention_terminal
Check	source IN ('invited', 'walk-in')	ck_visits_source
Check	status IN ('pending', 'security-review', 'scheduled', 'checked-in', 'temporarily-out', 'checked-out', 'no-show', 'cancelled', 'rejected')	ck_visits_status
Index	btree (host_id)	ix_visits_host_id
Index	btree (host_id, starts_at DESC)	ix_visits_host_start
Index	btree (site_id) WHERE status IN ('checked-in', 'temporarily-out')	ix_visits_on_site
Index	btree (site_id, status)	ix_visits_site_status
Index	btree (ends_at) WHERE status IN ('checked-in', 'temporarily-out')	ix_visits_overdue
Index	btree (retention_eligible_at) WHERE (retention_eligible_at IS NOT NULL) AND (anonymized_at IS NULL)	ix_visits_retention_pending
Index	btree (site_id)	ix_visits_site_id
Index	btree (site_id, status, starts_at)	ix_visits_site_status_start
Index	btree (visitor_person_id, starts_at DESC)	ix_visits_visitor_start
Trigger	reject_inactive_host()	trg_visits_active_host
Trigger	set_updated_at()	trg_visits_updated_at

Application rule. Which status changes are allowed (the visit state machine) is intended to be handled by the application; the database restricts the allowed values and requires check-in and check-out times for the statuses that need them.

2.22 visit_events

VISITS

PURPOSE Keeps a step-by-step history of each visit, such as approval, check-in, stepping out and check-out. Entries are only ever added, so the full history is preserved.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Required	Visit the event belongs to.
event_type	TEXT	Required	e.g. created, approved, check_in, step_out, reentry, check_out.
from_status	TEXT	Optional	Status before.
to_status	TEXT	Optional	Status after.
occurred_at	TIMESTAMPTZ	Required · default `now()`	When it happened.
recorded_at	TIMESTAMPTZ	Required · default `now()`	When it was recorded.
actor_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that acted, if any.
device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that acted, if any.
gate_label	VARCHAR(120)	Optional	Entry point label.
pass_id	UUID	FK → passes *on delete restrict* Optional	Pass involved, if any.
reason	VARCHAR(500)	Optional	Reason given.
metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).

CONSTRAINT	DEFINITION	NAME
Check	`event_type IN ('created', 'approved', 'rejected', 'security_review_requested', 'security_approved', 'security_denied', 'pass_issued', 'pass_revoked', 'check_in', 'step_out', 'reentry', 'check_out', 'cancelled', 'overdue_escalated', 'no_show', 'manual_correction', 'reconciled', 'identity_merged')`	`ck_visit_events_type`
Index	`btree (visit_id, occurred_at)`	`ix_visit_events_visit`
Trigger	`reject_history_change()`	`trg_visit_events_append_only`

2.23 passes

VISITS

PURPOSE Stores the QR pass issued for a visit and how long it is valid. Only a secure fingerprint of the QR code is used to look it up, so the code itself reveals nothing about the visitor.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Unique Required	Visit the pass belongs to.
token_hash	VARCHAR(64)	Unique Required	SHA-256 of the QR token, used for lookup.
token_ciphertext	TEXT	Optional	Encrypted copy of the token while the pass is active.
status	TEXT	Required · default `'active'`	active, revoked, expired or consumed.
usage	TEXT	Required · default `'multi_entry'`	single_entry or multi_entry.
valid_from	TIMESTAMPTZ	Required	Start of validity.
expires_at	TIMESTAMPTZ	Required	End of validity.
issued_by_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that issued it.
issued_by_device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that issued it.
issued_at	TIMESTAMPTZ	Required · default `now()`	When issued.
revoked_at	TIMESTAMPTZ	Optional	When revoked.
revoked_by	UUID	FK → users *on delete restrict* Optional	Account that revoked it.
revoke_reason	VARCHAR(500)	Optional	Reason given.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	`(status = 'active') OR (token_ciphertext IS NULL)`	`ck_passes_ciphertext_only_while_active`

CONSTRAINT	DEFINITION	NAME
Check	(status = 'revoked') = (revoked_at IS NOT NULL)	ck_passes_revoked_at
Check	status IN ('active', 'revoked', 'expired', 'consumed')	ck_passes_status
Check	usage IN ('single_entry', 'multi_entry')	ck_passes_usage
Check	expires_at > valid_from	ck_passes_validity
Index	btree (expires_at) WHERE status = 'active'	ix_passes_expiry_active
Trigger	set_updated_at()	trg_passes_updated_at

Application rule. Token generation, SHA-256 hashing and encryption are intended to be handled by the application.

2.24 pass_scans

VISITS

PURPOSE Records every time a pass is scanned or checked, and whether it was accepted or why it was refused. This helps to spot misuse and investigate entry problems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Sequential identifier.
pass_id	UUID	FK → passes *on delete restrict* Optional	Pass scanned (empty for unknown tokens).
site_id	UUID	FK → sites *on delete restrict* Optional	Site the row belongs to.
device_id	UUID	FK → devices *on delete restrict* Optional	Kiosk that scanned.
actor_user_id	UUID	FK → users *on delete restrict* Optional	Staff account that verified.
action	TEXT	Required	verify, check_in or check_out.
outcome	TEXT	Required	accepted, or the rejection reason.
scanned_at	TIMESTAMPTZ	Required · default `now()`	When it happened.

CONSTRAINT	DEFINITION	NAME
Check	action IN ('verify', 'check_in', 'check_out')	ck_pass_scans_action
Check	(device_id IS NOT NULL) OR (actor_user_id IS NOT NULL)	ck_pass_scans_actor

CONSTRAINT	DEFINITION	NAME
Check	`outcome IN ('accepted', 'malformed', 'unknown', 'expired', 'not_yet_valid', 'revoked', 'already_used', 'visit_cancelled', 'visit_completed', 'not_approved', 'wrong_site', 'not_eligible', 'security_pending', 'blacklisted')`	`ck_pass_scans_outcome`
Index	`btree (pass_id, scanned_at)`	`ix_pass_scans_pass`
Index	`btree (site_id, scanned_at) WHERE outcome <> 'accepted'`	`ix_pass_scans_site_rejected`
Trigger	`reject_history_change()`	`trg_pass_scans_append_only`

2.25 idempotency_keys

VISITS

PURPOSE Remembers recent requests so that a request sent twice, for example after a network drop at a kiosk, creates only one visit. Entries are kept for about a day.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
scope	VARCHAR(80)	PK *composite* Required	Caller scope (user or device).
key	VARCHAR(120)	PK *composite* Required	Client-supplied key.
request_hash	VARCHAR(64)	Required	Hash of the request body.
response_status	INTEGER	Optional	Stored HTTP status.
response_body	JSONB	Optional	Stored response (JSON).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
expires_at	TIMESTAMPTZ	Required	When the key may be removed.

CONSTRAINT	DEFINITION	NAME
Index	`btree (expires_at)`	`ix_idempotency_keys_expires`

Application rule. Replaying stored responses is handled by the application; expired keys are intended to be removed by the retention job.

2.26 blacklist_entries

SECURITY

PURPOSE Lists people who should be refused entry or need security approval, with the reason and how long the restriction lasts. Visitors are checked against this list.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the restriction applies to; empty means all sites.
person_id	UUID	FK → people *on delete restrict* Optional	Linked person, if known.
match_name	VARCHAR(160)	Optional	Name to match.
match_email	VARCHAR(320)	Optional	Email to match (lowercase).
match_phone_digits	VARCHAR(40)	Optional	Last 10 digits of a phone number.
reason_category	TEXT	Required	safety, theft, harassment, trespass, legal_order, policy_violation or other.
reason_detail	VARCHAR(1000)	Optional	Restricted detail.
severity	TEXT	Required	deny or require_approval.
effective_from	TIMESTAMPTZ	Required · default `now()`	Start of the restriction.
effective_until	TIMESTAMPTZ	Optional	End, if any.
review_due_at	TIMESTAMPTZ	Required	Date the record must be reviewed.
status	TEXT	Required · default `'active'`	active, expired or lifted.
created_by	UUID	FK → users *on delete restrict* Required	Account that created it.
lifted_by	UUID	FK → users *on delete restrict* Optional	Account that lifted it.
lifted_at	TIMESTAMPTZ	Optional	When lifted.
lift_reason	VARCHAR(500)	Optional	Reason given.
version	INTEGER	Required · default `1`	Optimistic-locking counter, incremented by the application on each update.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Check	(match_email IS NULL) OR (match_email = lower((match_email)))	ck_blacklist_email_lowercase
Check	(person_id IS NOT NULL) OR (match_email IS NOT NULL) OR (match_phone_digits IS NOT NULL) OR (match_name IS NOT NULL)	ck_blacklist_identifier
Check	(status = 'lifted') = (lifted_at IS NOT NULL)	ck_blacklist_lifted
Check	(effective_until IS NULL) OR (effective_until > effective_from)	ck_blacklist_period
Check	reason_category IN ('safety', 'theft', 'harassment', 'trespass', 'legal_order', 'policy_violation', 'other')	ck_blacklist_reason
Check	severity IN ('deny', 'require_approval')	ck_blacklist_severity
Check	status IN ('active', 'expired', 'lifted')	ck_blacklist_status
Index	btree (match_email) WHERE status = 'active'	ix_blacklist_email
Index	btree (person_id) WHERE status = 'active'	ix_blacklist_person
Index	btree (match_phone_digits) WHERE status = 'active'	ix_blacklist_phone
Index	btree (review_due_at) WHERE status = 'active'	ix_blacklist_review_due
Trigger	set_updated_at()	trg_blacklist_entries_updated_at

Application rule. Matching visitors against these records and re-checking upcoming visits are intended to be handled by the application, which also limits the review date to 12 months.

2.27 blacklist_matches

SECURITY

PURPOSE Records when a visitor appears to match a restricted-entry record and how certain the match is. This gives security staff what they need to make a decision.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
blacklist_entry_id	UUID	FK → blacklist_entries *on delete restrict* Required	Record matched.
visit_id	UUID	FK → visits *on delete restrict* Required	Visit that matched.
match_basis	TEXT	Required	person, email, phone or name.
confidence	TEXT	Required	confirmed, uncertain or rejected.
reviewed_by	UUID	FK → users *on delete restrict* Optional	Reviewer.
reviewed_at	TIMESTAMPTZ	Optional	When reviewed.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

CONSTRAINT	DEFINITION	NAME
Unique	`blacklist_entry_id, visit_id`	`uq_blacklist_match_entry_visit`
Check	`match_basis IN ('person', 'email', 'phone', 'name')`	`ck_blacklist_match_basis`
Check	`confidence IN ('confirmed', 'uncertain', 'rejected')`	`ck_blacklist_match_confidence`
Check	`(reviewed_by IS NULL) = (reviewed_at IS NULL)`	`ck_blacklist_match_review`
Index	`btree (visit_id)`	`ix_blacklist_matches_visit`

2.28 security_approvals

SECURITY

PURPOSE Records the security team's decision about a visitor who matched a restricted-entry record. A flagged visit cannot go ahead until this decision is made.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
visit_id	UUID	FK → visits *on delete restrict* Unique where status = 'pending' Required	Visit under review.
status	TEXT	Required · default `'pending'`	pending, approved, denied or superseded.
requested_at	TIMESTAMPTZ	Required · default `now()`	When review started.
decided_by	UUID	FK → users *on delete restrict* Optional	Decision maker.
decided_at	TIMESTAMPTZ	Optional	When decided.
decision_reason	VARCHAR(500)	Optional	Reason given.

CONSTRAINT	DEFINITION	NAME
Unique index	`visit_id WHERE status = 'pending'`	`uq_security_approval_pending`
Check	`status IN ('pending', 'superseded') OR ((decided_by IS NOT NULL) AND (decided_at IS NOT NULL) AND (decision_reason IS NOT NULL))`	`ck_security_approval_decision`
Check	`status IN ('pending', 'approved', 'denied', 'superseded')`	`ck_security_approval_status`
Index	`btree (status, requested_at)`	`ix_security_approvals_status`

Application rule. The application is intended to prevent a host from making the security decision for their own visitor.

2.29 audit_logs

AUDIT

PURPOSE Keeps a permanent record of important actions, such as who approved a visit or changed an account. Entries cannot be edited, so the record can be relied on in an investigation.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
actor_user_id	UUID	FK → users *on delete restrict* Optional	User who performed the action, if any.
action	VARCHAR(80)	Required	e.g. visit.approved, blacklist.created.
created_at	TIMESTAMPTZ	Required · default `now()`	When the action occurred.
actor_device_id	UUID	FK → devices *on delete restrict* Optional	Device that performed it, if any.
entity_type	VARCHAR(40)	Required	Kind of record affected.
entity_id	UUID	Optional	Identifier of the affected record.
site_id	UUID	Optional	Site involved, if any.
correlation_id	VARCHAR(64)	Optional	Request id (X-Request-ID).
ip	INET	Optional	Client IP address.
metadata	JSONB	Required · default `'{}'`	Structured, non-sensitive details (JSON).

CONSTRAINT	DEFINITION	NAME
Index	btree (action, created_at DESC)	ix_audit_logs_action
Index	btree (actor_user_id, created_at DESC)	ix_audit_logs_actor
Index	btree (entity_type, entity_id)	ix_audit_logs_entity
Index	btree (created_at DESC)	ix_audit_logs_created
Index	btree (site_id, created_at DESC) WHERE site_id IS NOT NULL	ix_audit_logs_site_created
Trigger	reject_history_change()	trg_audit_logs_append_only

Application rule. Rows are intended to be written only through the application's audit service, which refuses anything that looks like a password, token or pass code.

2.30 notifications

AUDIT

PURPOSE Stores messages sent to staff, such as "your visitor has arrived", and whether each was delivered. It also lets each person see their own notifications in the dashboard.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required	Unique identifier.
recipient_id	UUID	FK → users *on delete restrict* Required	Staff account notified.
visit_id	UUID	FK → visits *on delete set null* Optional	Visit concerned, if any.
title	VARCHAR(180)	Required	Title.
message	VARCHAR(500)	Required	Message text.
read	BOOLEAN	Required · default `false`	Whether the recipient has read it.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
kind	VARCHAR(40)	Required · default `'general'`	e.g. arrival, overdue, security_review.
channel	TEXT	Required · default `'in_app'`	in_app, email or sms.
status	TEXT	Required · default `'delivered'`	queued, attempting, accepted, delivered, failed or unknown.
read_at	TIMESTAMPTZ	Optional	When read.
delivered_at	TIMESTAMPTZ	Optional	When delivered.

CONSTRAINT	DEFINITION	NAME
Check	`channel IN ('in_app', 'email', 'sms')`	`ck_notifications_channel`
Check	`status IN ('queued', 'attempting', 'accepted', 'delivered', 'failed', 'unknown')`	`ck_notifications_status`
Index	`btree (created_at) WHERE status IN ('queued', 'attempting')`	`ix_notifications_queued`
Index	`btree (recipient_id)`	`ix_notifications_recipient_id`
Index	`btree (recipient_id, read)`	`ix_notifications_recipient_read`

Application rule. Sending email copies through the outbox is intended to be handled by the application.

2.31 notification_attempts

AUDIT

PURPOSE Records each attempt to send an email or text message and its result. This shows whether a message got through and helps to diagnose delivery problems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Sequential identifier.
notification_id	UUID	FK → notifications *on delete cascade* Required	Notification attempted.
attempted_at	TIMESTAMPTZ	Required · default `now()`	When.
provider	VARCHAR(40)	Optional	Delivery provider.
provider_message_id	VARCHAR(200)	Optional	Provider's message id.
outcome	TEXT	Required	accepted, delivered, failed or unknown.
error_code	VARCHAR(80)	Optional	Error, if failed.

CONSTRAINT	DEFINITION	NAME
Check	`outcome IN ('accepted', 'delivered', 'failed', 'unknown')`	`ck_notification_attempt_outcome`
Index	`btree (notification_id)`	`ix_notification_attempts_notification`

2.32 outbox_events

AUDIT

PURPOSE Holds a queue of follow-up work created when a record is saved, such as emails to send. This makes sure the work happens reliably even if a delivery fails, and it can also feed changes to other systems.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	BIGINT	PK Identity (always)	Sequential id; also the sync cursor.
event_id	UUID	Unique Required · default `gen_random_uuid()`	Stable event identifier.
aggregate_type	VARCHAR(40)	Required	Kind of record changed.
aggregate_id	UUID	Required	Record changed.
aggregate_version	INTEGER	Optional	Record version after the change.
event_type	VARCHAR(60)	Required	e.g. visit.status_changed.
payload	JSONB	Required · default `'{}'`	Identifiers and states only (JSON).
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.

available_at	TIMESTAMPTZ	Required · default `now()`	Earliest next processing time.
attempts	INTEGER	Required · default `0`	Failed processing attempts.
last_error	VARCHAR(500)	Optional	Last failure.
processed_at	TIMESTAMPTZ	Optional	When processed.
dead_lettered_at	TIMESTAMPTZ	Optional	When given up on.

CONSTRAINT	DEFINITION	NAME
Check	`NOT ((processed_at IS NOT NULL) AND (dead_lettered_at IS NOT NULL))`	`ck_outbox_single_outcome`
Index	`btree (aggregate_type, aggregate_id, id)`	`ix_outbox_aggregate`
Index	`btree (dead_lettered_at) WHERE dead_lettered_at IS NOT NULL`	`ix_outbox_dead`
Index	`btree (available_at, id) WHERE (processed_at IS NULL) AND (dead_lettered_at IS NULL)`	`ix_outbox_pending`

Application rule. Processing, retries and giving up on repeatedly failing events are intended to be handled by a background worker.

2.33 retention_policies

OPS

PURPOSE Sets how long each kind of record is kept and whether it is deleted or anonymised afterwards. A system-wide default can be adjusted for individual sites.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
site_id	UUID	FK → sites *on delete restrict* Optional	Site the rule applies to; empty for the system-wide default.
record_class	TEXT	Required	ordinary_visit, security_visit, pass_scan, notification, auth_event, audit_log, outbox_event or job_run.
retain_months	INTEGER	Required	Months to keep.
action	TEXT	Required	delete or anonymize.
approved_by	UUID	FK → users *on delete restrict* Optional	Account that set it.
approved_at	TIMESTAMPTZ	Optional	When set.
notes	VARCHAR(500)	Optional	Notes.
created_at	TIMESTAMPTZ	Required · default `now()`	When the row was created.
updated_at	TIMESTAMPTZ	Required · default `now()`	Last change; maintained by trigger.

CONSTRAINT	DEFINITION	NAME
Unique index	`site_id, record_class (nulls not distinct)`	`uq_retention_policy_scope`
Check	`action IN ('delete', 'anonymize')`	`ck_retention_action`
Check	`(retain_months ⩾ 1) AND (retain_months ⩽ 120)`	`ck_retention_months`
Check	`record_class IN ('ordinary_visit', 'security_visit', 'pass_scan', 'notification', 'auth_event', 'audit_log', 'outbox_event', 'job_run')`	`ck_retention_record_class`
Trigger	`set_updated_at()`	`trg_retention_policies_updated_at`

Application rule. The retention job is intended to apply the most specific rule: a site rule if one exists, otherwise the system-wide default.

2.34 retention_holds

OPS

PURPOSE Marks records that must be kept beyond the normal retention period, for example during a legal case or an investigation. Each hold needs a reason and a review date.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
entity_type	TEXT	Required	visit or person.
entity_id	UUID	Required	Record held.
hold_type	TEXT	Required	legal, investigation or security_incident.
reason	VARCHAR(500)	Required	Reason given.
case_reference	VARCHAR(120)	Optional	Case or matter reference.
placed_by	UUID	FK → users *on delete restrict* Required	Account that placed it.
placed_at	TIMESTAMPTZ	Required · default `now()`	When placed.
review_due_at	TIMESTAMPTZ	Required	Review date.
released_by	UUID	FK → users *on delete restrict* Optional	Account that released it.
released_at	TIMESTAMPTZ	Optional	When released.
release_reason	VARCHAR(500)	Optional	Reason given.

CONSTRAINT	DEFINITION	NAME
Check	`(hold_type = 'security_incident') OR (case_reference IS NOT NULL)`	`ck_retention_hold_case_reference`
Check	`entity_type IN ('visit', 'person')`	`ck_retention_hold_entity`
Check	`(released_at IS NULL) = (released_by IS NULL)`	`ck_retention_hold_release`
Check	`(review_due_at > placed_at) AND (review_due_at ⩽ (placed_at + '1 year'))`	`ck_retention_hold_review`
Check	`hold_type IN ('legal', 'investigation', 'security_incident')`	`ck_retention_hold_type`
Index	`btree (entity_type, entity_id) WHERE released_at IS NULL`	`ix_retention_holds_active`

Application rule. Review reminders are intended to be sent by the retention job.

2.35 visit_daily_stats

OPS

PURPOSE Keeps daily visit totals without any personal details. This allows reporting on past visit numbers after the individual visit records have been anonymised.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
site_id	UUID	PK *composite* FK → sites *on delete restrict* Required	Site the row belongs to.
day	DATE	PK *composite* Required	Calendar day.
source	TEXT	PK *composite* Required	invited or walk-in.
category	TEXT	PK *composite* Required	Visit category.
final_status	TEXT	PK *composite* Required	Status when anonymized.
visits	INTEGER	Required · default `0`	Number of visits.
total_minutes_on_site	BIGINT	Required · default `'0'`	Total minutes between check-in and check-out.

Application rule. Rows are intended to be written by the retention job as visits are anonymised.

2.36 job_runs

OPS

PURPOSE Records each run of a background task, such as the nightly clean-up, and whether it succeeded. This helps to confirm that scheduled work is happening.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
job_name	VARCHAR(60)	Required	Job that ran.
started_at	TIMESTAMPTZ	Required · default `now()`	Start.
finished_at	TIMESTAMPTZ	Optional	End.
status	TEXT	Required · default `'running'`	running, succeeded, failed or partial.
dry_run	BOOLEAN	Required · default `false`	Whether changes were rolled back.
counts	JSONB	Required · default `'{}'`	What the run did (JSON).
error	VARCHAR(1000)	Optional	Failure message.

CONSTRAINT	DEFINITION	NAME
Check	`(status = 'running') = (finished_at IS NULL)`	`ck_job_runs_finished`
Check	`status IN ('running', 'succeeded', 'failed', 'partial')`	`ck_job_runs_status`
Index	`btree (job_name, started_at DESC)`	`ix_job_runs_name_started`

Application rule. Intended to be written by the background worker.

2.37 backup_runs

OPS

PURPOSE Records each database backup and each test restore, and whether it succeeded. This gives evidence that usable backups exist.

COLUMN	DATA TYPE	KEY / RULE	WHAT IT STORES
id	UUID	PK Required · default `gen_random_uuid()`	Unique identifier.
kind	TEXT	Required	full or restore_test.
started_at	TIMESTAMPTZ	Required · default `now()`	Start.
finished_at	TIMESTAMPTZ	Optional	End.
status	TEXT	Required · default `'running'`	running, succeeded or failed.
location	VARCHAR(500)	Optional	Backup file path.
size_bytes	BIGINT	Optional	File size.
checksum	VARCHAR(64)	Optional	SHA-256 of the unencrypted dump.
details	JSONB	Required · default `'{}'`	Checks and counts (JSON).
error	VARCHAR(1000)	Optional	Failure message.

CONSTRAINT	DEFINITION	NAME
Check	`kind IN ('full', 'restore_test')`	ck_backup_runs_kind
Check	`status IN ('running', 'succeeded', 'failed')`	ck_backup_runs_status
Index	`btree (kind, started_at DESC)`	ix_backup_runs_kind_started

Application rule. Intended to be written by the backup service and its restore tests.

3 Data types and key terms

The PostgreSQL types used in the proposed design, and the constraint terms used in the tables above.

Data types used

TYPE	COLUMNS	TYPICAL USE
TIMESTAMPTZ	100	Dates and times, stored in UTC.
UUID	99	Identifiers and references.
VARCHAR	64	Names, emails and labels with a length limit.
TEXT	44	Statuses, codes and free text without a length limit.
INTEGER	20	Counts, versions and settings.

TYPE	COLUMNS	TYPICAL USE
`BOOLEAN`	11	True/false flags.
`JSONB`	9	Structured metadata and settings.
`SMALLINT`	6	Role and permission identifiers.
`BIGINT`	6	Sequential ids of append-only logs, and byte sizes.
`INET`	4	IP addresses.
`DATE`	1	Calendar days in statistics.

Key terms

TERM	MEANING
PK **Primary key**	Identifies one row uniquely.
FK **Foreign key**	Links a row to a record in another table; the database rejects links to missing rows.
Unique / unique index	Prevents duplicate values. A partial unique index applies only to rows matching its WHERE clause.
Check	A condition every row must satisfy.
Generated column	Computed by the database from other columns; cannot be written.
Restrict / cascade / set null	What happens to referencing rows when a referenced row is deleted: block the delete, delete them too, or clear the reference.
Trigger	Database code that runs on insert/update/delete (Section 5).

4 Security notes: passwords and pass tokens

4.1 Password hashing

TOPIC	PROPOSED APPROACH	ENFORCED BY
Storage	Hashes are kept in `user_credentials.password_hash`, separate from `users` and `people`.	Database (separate table)
Algorithm	Argon2id with a per-password salt, using a maintained library (pwdlib). Hashes made with outdated parameters are upgraded at the next sign-in.	Application
Plaintext guard	Check `ck_user_credentials_argon2id` rejects any value that does not start with `$argon2id$`.	Database
Reset links	Only a keyed hash of each reset token is stored (`password_reset_requests.token_hash`, unique), with at most one live request per user.	Database (unique indexes); hashing by application

4.2 QR pass tokens and device keys

> **Pass tokens are different from passwords.** The QR is intended to hold a random 256-bit token and no personal data. `passes.token_hash` stores its SHA-256 for lookup (unique). `passes.token_ciphertext` holds an encrypted copy so staff can re-display an active pass; check `ck_passes_ciphertext_only_while_active` requires it to be empty once the pass is no longer active. Each visit has at most one pass (unique `visit_id`). Kiosk device keys and pairing codes are stored only as keyed hashes in `devices`.

5 Database functions, triggers and seed data

Rules the database itself is intended to enforce beyond keys and checks, the extensions required, and the reference data to be loaded when the database is created.

FUNCTION	ATTACHED TO	WHAT IT ENFORCES
enforce_user_role_scope()	user_roles	Rejects a global role assigned to a site, and a site-level role assigned without a site (based on `roles.scope`).
reject_history_change()	audit_logs, authentication_events, pass_scans, visit_events	Makes the table append-only: UPDATE and DELETE fail unless the transaction sets `vms.allow_history_changes = on`, which is intended for use only by the retention job.
reject_inactive_host()	invitations, visits	Rejects new visits or invitations (or a change of host) for a host whose status is not active.
set_updated_at()	blacklist_entries, departments, devices, gates, hosts, invitations, passes, people, person_contacts, retention_policies, site_settings, sites, user_credentials, users, visitor_profiles, visits	Sets `updated_at` to now() on every update.

Extensions: `pg_trgm` (trigram indexes for name and employee-ID search, and similarity matching), plus `plpgsql`.

Proposed roles

A global role applies to every site; `global_or_site` roles can cover every site or selected sites; site roles always name their sites.

CODE	SCOPE	PRIORITY
super-admin	global	100
site-admin	global_or_site	80
receptionist	site	40
host	site	20

Proposed permissions

PERMISSION	GRANTED TO
system.manage	super-admin
site.manage	super-admin, site-admin
user.read	super-admin, site-admin
user.manage	super-admin, site-admin
user.reset_password	super-admin, site-admin
host.manage	super-admin, site-admin
host.read	super-admin, site-admin, receptionist, host
visit.read	super-admin, site-admin, receptionist, host
visit.create	super-admin, site-admin, host
visit.update	super-admin, site-admin, host
visit.approve	super-admin, site-admin, host
visit.reject	super-admin, site-admin
pass.revoke	super-admin, site-admin, host
visitor.update	super-admin, site-admin, receptionist
visitor.deactivate	super-admin, site-admin
walk_in.register	super-admin, site-admin, receptionist
audit.read	super-admin, site-admin, receptionist, host
visit.check_in	super-admin, site-admin, receptionist
visitor.merge	super-admin, site-admin
device.manage	super-admin, site-admin
blacklist.manage	super-admin, site-admin
security.approve	super-admin, site-admin
retention.hold	super-admin, site-admin

Default retention policies

System-wide defaults; any of them can be overridden for an individual site.

RECORD CLASS	MONTHS	ACTION
audit_log	24	delete
auth_event	12	delete
job_run	6	delete

RECORD CLASS	MONTHS	ACTION
notification	3	delete
ordinary_visit	5	anonymize
outbox_event	1	delete
pass_scan	5	delete
security_visit	24	anonymize

SUGGESTED NEXT STEP

Review the proposed tables, constraints and default retention periods with the team, and confirm the remaining choices, such as how notifications are delivered and how production backups are taken, before the schema is settled.